

HealthClaimCore Z6 Validation Report

Decision: PASS WITH OBSERVATIONS

The isolated HealthClaimCore module verified, all tests passed, and both benchmarks completed. Trusted authority keys, registry roots, and policy commitments still need to be anchored by a verifier or authoritative state outside this circuit.

Scope and changes

I read the attached implementation plan and integrated result before validation. This report documents validation only. No project files were changed for the validation, and Hardened and Gateway work was not started.

Environment and reproducibility

- OS/architecture: Ubuntu Linux, linux/amd64, kernel 6.17.0-1032-oem
- CPU: AMD Ryzen 7 PRO 8840HS, 8 cores / 16 threads
- RAM: 27 GiB reported by the host
- Go: go1.25.8; module declares Go 1.25.7 and toolchain go1.25.8
- Dependencies: gnark v0.16.3; gnark-crypto v0.21.0
- go mod verify: all modules verified

The first test attempt could not write to the default Go build cache because it is read-only in this environment. Tests and benchmarks passed after setting GOCACHE=/tmp/healthclaim-go-cache.

Commands run from zkp/healthclaimcore:

```
go version
go mod verify
GOCACHE=/tmp/healthclaim-go-cache go test -count=1 -v ./...
GOCACHE=/tmp/healthclaim-go-cache go test -count=1 -run '^$' -bench
'BenchmarkProof(Generation|Verification)$' -benchtime=10x -benchmem
```

Exact circuit relation

The circuit implements a BN254 Groth16 relation with an 8-level Merkle path and a fixed four-entry diagnosis set:

1. `keyCommitment = Poseidon2(domainSecret, patientSecret).`
2. `leaf = Poseidon2(domainRecord, diagnosis, labValue, policyID, coverageCeiling, keyCommitment).`
3. Verify the authority EdDSA signature over `leaf`, using MiMC as the signature challenge hash.
4. Recompute the Poseidon2 policy commitment from the policy ID, lab bounds, maximum claim, and four covered diagnoses.
5. Constrain every Merkle direction bit to Boolean, compute the path, and require its result to equal the public registry root.

6. Require diagnosis membership in exactly one of the four committed entries.
7. Enforce explicit 64-bit ranges and `labMin <= labValue <= labMax`.
8. Enforce `coverageCeiling <= policyMaxClaim` and `claimAmount <= coverageCeiling`.
9. Derive the record index from the Merkle direction bits and require `nullifier = Poseidon2(domainNullifier, patientSecret, index)`.

Inputs

Public inputs (6): registry root, policy commitment, claim amount, authority public key X and Y coordinates, and nullifier.

Private witness (31): diagnosis, lab value, policy ID, coverage ceiling, patient secret, lab minimum and maximum, policy maximum claim, four covered diagnoses, signature, eight Merkle siblings, and eight Merkle direction bits.

Constraints

- R1CS constraints: **19,657**
- Public variables: **6** (excluding the constant-one variable)
- Secret variables: **31**

Test results

`go test -count=1 -v ./...`: **PASS**. Six top-level tests and 21 subtests passed; none failed or skipped.

Coverage included deterministic vectors; native EdDSA message parity; Merkle direction, index, and nullifier semantics; authority-key anchoring behavior; invalid signatures; off-curve authority and signature points; wrong roots and policy commitments; diagnosis exclusion; lab bounds; claim and coverage limits; non-Boolean Merkle directions; wrong nullifiers; and proof rejection after mutation of each of the six public inputs.

Benchmark results

`go test -count=1 -run '^$' -bench 'BenchmarkProof(Generation|Verification)$' -benchtime=10x -benchmem`: **PASS**.

Benchmark	Time	Bytes/op	Allocs/op
Proof generation	64.80 ms/op	21,163,069	2,644
Proof verification	0.635 ms/op	30,384	165

Deviations from the PDF

HealthClaimCore intentionally omits the planned Hardened-stage context inputs: root version, challenge, recipient context, and domain separator binding. It therefore does not establish freshness, requester binding, or version-window acceptance.

Security concerns and unresolved issues

The circuit accepts its authority public key, registry root, and policy commitment as public inputs; it does not itself prove that they came from trusted state. A test confirms that a self-selected attacker key with a matching signature can satisfy the relation. The verifier must anchor the key and public state before treating a proof as authorization. Replay prevention also requires the external system to reject nullifiers that have already been used.

These are integration and planned Hardened-phase requirements, not failures of the tested Core relation. Gateway, Fabric, IPFS, Docker, networking, and the legacy gnark implementation were not modified.